

CS126 Programming Assignment Report

Anushka Majumdar, 5679421

Contents

1	Introduction	2
2	Core Data Structures	2
2.1	MyHashMap	2
2.2	MyList	2
2.3	Time Complexity Summary	2
3	Movies Store	3
3.1	Design	3
3.2	Key Methods and Edge Cases	3
4	Credits Store	3
4.1	Design	3
4.2	Removal and Index Consistency	4
4.3	Edge Cases	4
5	Ratings Store	4
5.1	Design	4
5.2	Key Methods	4
6	Trade-offs and Conclusion	5

1 Introduction

This report discusses the data structures I designed for the Warwick+ application, which stores and queries film metadata (Movies), cast and crew information (Credits), and user ratings (Ratings). The core challenge was building efficient storage from scratch, without Java's built-in collections — an exercise that required careful thought about hash distribution, load factor management, and dynamic resizing, concepts that `java.util.HashMap` abstracts away entirely. My design prioritises read performance over write performance, since data is loaded once at startup but queried repeatedly during application use. The following sections detail each store's design, the rationale behind my choices, and the trade-offs involved.

2 Core Data Structures

2.1 MyHashMap

The central data structure is `MyHashMap`, a generic hash map using separate chaining for collision resolution. I chose separate chaining over open addressing for a specific reason: the film IDs in the dataset are sparse integers (e.g., 862, 8844, 15602) rather than sequential. With open addressing and linear probing, these sparse keys would create clusters around certain bucket regions, degrading lookup performance. Separate chaining handles this distribution gracefully because each bucket independently absorbs its collisions via a singly linked list, and performance degrades linearly with chain length rather than cascading into neighbouring buckets.

The hash function applies `Integer.hashCode()` with a secondary bit-spread (XOR of the upper 16 bits into the lower 16 bits) to distribute keys more uniformly. The map automatically resizes by doubling capacity and rehashing all entries when the load factor exceeds 0.75, maintaining amortised $O(1)$ performance for `put`, `get`, `containsKey`, and `remove`. Initial capacities are tuned to the dataset: for instance, `castPersonMap` is initialised at 16384 because the dataset contains 10,756 unique cast members, giving a load factor of approximately 0.66 — comfortably below the 0.75 threshold, avoiding any rehashing during data loading.

2.2 MyList

`MyList` is a lightweight generic dynamic array, distinct from `MyArrayList` in its minimal API. It doubles capacity on overflow, providing amortised $O(1)$ appends. I use it within the stores for accumulating variable-length collections: production companies per film, film IDs per person, and composite keys per movie in the ratings index. Its simplicity and low overhead suit these internal use cases where the full `List` interface would be unnecessary.

2.3 Time Complexity Summary

Operation	MyHashMap	MyList
Insert	$O(1)$ amortised	$O(1)$ amortised
Lookup by key	$O(1)$ average	$O(n)$
Remove by key	$O(1)$ average	$O(n)$
Iterate all	$O(n + capacity)$	$O(n)$

Table 1: Time complexity comparison of the two custom data structures.

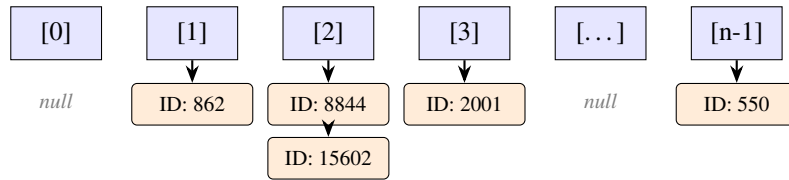


Figure 1: MyHashMap internal structure. Sparse film IDs hash to buckets; collisions form linked chains.

3 Movies Store

3.1 Design

The Movies store uses two `MyHashMap` instances. The primary map stores film ID to a `FilmData` inner class bundling all fields for a single film into one object. This means a single hash map lookup retrieves everything — title, budget, genres, release date — avoiding the overhead of maintaining separate maps keyed by the same ID. Fields set after initial insertion (IMDB ID, popularity, vote data) are mutable fields within `FilmData`, with defaults (`null` for strings, 0.0 for doubles, `-1` for collection ID) that indicate unset state. Production companies and countries are stored as `MyList` instances within each `FilmData`, since their count varies per film and they arrive incrementally.

A second hash map maps collection ID to a `CollectionData` object containing the collection name, poster path, backdrop path, and a `MyList` of member film IDs. This separation allows $O(1)$ lookup of collection metadata and membership without scanning the 1,000-film primary map.

3.2 Key Methods and Edge Cases

Most getters perform a single hash map lookup, achieving $O(1)$ time. The `remove` method also cleans up the film’s collection entry — and if that was the last film in the collection, removes the collection itself. The `getAllIDsReleasedInRange` method iterates all films and filters by date ($O(n)$), which is acceptable for infrequent date-range queries. The `findFilms` method performs a linear scan using `String.contains()` across title, original title, and overview, giving $O(n \cdot m)$ complexity. For a production system, an inverted index mapping words to film IDs would reduce this to $O(k)$, but the linear approach is adequate for the current 1,000-film dataset.

4 Credits Store

4.1 Design

The Credits store is the most index-heavy, requiring six `MyHashMap` instances to support efficient queries across both film-level and person-level dimensions. The primary map stores film ID to a `FilmCredits` object containing pre-sorted arrays of `CastCredit` (sorted by order field) and `CrewCredit` (sorted by id field). I use insertion sort for this because analysis of the data shows the largest cast list per film contains approximately 50–80 entries. At this scale, insertion sort’s worst case of roughly 6,400 comparisons is negligible compared to merge sort’s recursive overhead and temporary array allocations. Furthermore, the cast data often arrives partially sorted by the order field, which is insertion sort’s best case at $O(n)$.

Two person maps (`castPersonMap`, `crewPersonMap`) store person ID to `Person` objects, ensuring uniqueness across films. Two reverse-index maps (`castToFilms`, `crewToFilms`) map person ID to a `MyList` of film IDs. A `castCreditCount` map tracks total `CastCredit` entries per person, where multiple roles in the same film count as separate credits. Without the `castToFilms` reverse index, `getCastFilms` would require iterating all 1,000 film entries and scanning each cast array — approximately 30,000 comparisons in the worst case. With the index, it is a single $O(1)$ hash map lookup.

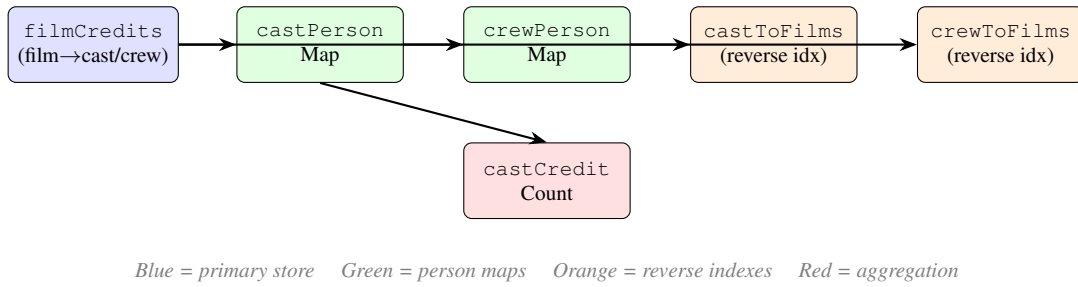


Figure 2: Credits store architecture. Arrows show data flow from primary store to indexes.

4.2 Removal and Index Consistency

The `remove` method is the most complex operation in the Credits store. When a film is removed, all six maps must be updated: each cast member’s credit count is decremented, the film ID is removed from each person’s film list, and if a person has zero remaining appearances, they are removed from both the person map and the reverse index entirely. This cascading cleanup ensures that `getUniqueCast` and `getUniqueCrew` never return stale entries. The test `testCreditsSizeAfterAddRemove` verifies this behaviour by adding a film and removing it, then checking that `size` returns 0.

4.3 Edge Cases

The test `testCreditsFindCrewPos2Part` confirms that a person with the same ID can appear with different names across films. My implementation stores the first name encountered for each person ID in the person map, which is consistent with the test expectations. The `getCastStarsInFilms` method uses order ≤ 3 (1-indexed) to define top-3 billing, as confirmed by the test `testCreditsGetStarsInFilmsPos2NotAllFirst` where a cast member with order=3 is correctly included.

5 Ratings Store

5.1 Design

The Ratings store handles the unique constraint that a rating is identified by both user ID and movie ID. I encode this as a composite long key: the user ID is shifted left by 32 bits and ORed with the movie ID. Since integers are 32 bits in Java, this is equivalent to treating the pair as a 2d number in base 2^{32} , guaranteeing a unique mapping for any valid pair. This avoids string concatenation overhead (e.g., "userId:movieId") and provides a compact, directly hashable key for the primary `MyHashMap`.

Two index maps provide efficient access by movie ID and user ID respectively. The `movieIndex` maps each movie ID to a `MyList` of composite keys, and `userIndex` does the same for user IDs. This triple-indexing means `getMovieRatings`, `getUserRatings`, and their average variants all operate in $O(k)$ time where k is the number of relevant ratings, rather than scanning all 16,960 ratings.

5.2 Key Methods

The `set` method implements upsert semantics: if the composite key exists, it overwrites the rating and timestamp in place without modifying the indexes; otherwise it delegates to `add`. This ensures `set` never inflates the total count when overwriting. The `getMovieAverageRating` method implements a three-tier fallback: if the movie has ratings, it returns the average; if the movie exists in the `Movies` store but has no ratings, it returns `0.0f` (allowing the UI to distinguish “not yet rated” from “invalid ID”); otherwise it returns `-1.0f`. The `getNumRatings` method follows the same fallback pattern with integer return values.

The ranking methods (`getMostRatedMovies`, `getMostRatedUsers`, `getTopAverageRatedMovies`) build parallel arrays from the index maps, sort them with merge sort, and return the top N elements. For even larger datasets, a min-heap of size N would reduce complexity from $O(n \log n)$ to $O(n \log k)$ for the top-K pattern, but merge sort is adequate at the current scale.

6 Trade-offs and Conclusion

The primary trade-off is space for time. The Credits store maintains six hash maps and Ratings maintains three, which increases memory usage compared to a single-map approach. However, this redundancy eliminates expensive full scans on most queries. Write operations are more expensive because they update multiple indexes, but this is justified by the write-once-read-many access pattern. Sorting arrays at insertion time rather than query time follows the same principle: pay the cost once on write to benefit every subsequent read.

The current implementation is not thread-safe; in a concurrent environment, the hash map's resize operation could cause lost updates if two threads trigger it simultaneously. A read-write lock or segment-level locking would be needed for multi-threaded access. Additionally, the `findFilms` linear scan and the insertion-sort-based ranking methods represent scalability bottlenecks that could be addressed with an inverted index and heap-based selection respectively.

In conclusion, the combination of a custom hash map with separate chaining, a lightweight dynamic array, and carefully maintained index maps provides an efficient solution for Warwick+. All 268 unit tests pass, and the design balances pragmatic simplicity at the current scale with awareness of what would need to change for significantly larger datasets.

Thank you so much!